PGF–JESPRIT, Current State of the `cikm` Codebase

A math-focused walkthrough of the supervisor's implementation and how it differs from the original JESPRIT prototype

Internal comparison note

May 29, 2026

Abstract

This note documents the current state of the supervisor's `cikm` repository (`src/pgf_jesprit`) and contrasts it, derivation by derivation, with the original JESPRIT prototype (`JESPRIT/src`). The shared backbone is unchanged: a finite product–Poisson mixture is sketched by sampling its empirical probability generating function (PGF) along complex lines, the samples are organised as a multi–snapshot sum–of–exponentials model, and a global–subspace joint–ESPRIT pipeline recovers the rate vectors and mixture weights. The supervisor's version adds five mathematically substantive ingredients on top of that backbone: (i) *damped* PGF queries that keep every evaluation point inside the closed unit disk, controlling the variance of the empirical PGF; (ii) a *non–unitary* (similarity) joint diagonalisation in place of the unitary Cardoso–Souloumiac Jacobi sweep; (iii) *constrained* parameter recovery (non–negative least squares for rates, simplex–constrained least squares for weights); (iv) a post–ESPRIT *refinement* that minimises the exact PGF least–squares loss by L–BFGS–B using the closed–form analytic gradient and a log/softmax reparametrisation (this replaces the earlier projected–gradient–descent loop); and (v) the *NB1* model, a one–parameter–per–coordinate negative–binomial family that sits between the finite Poisson mixture and a full Poisson–Gamma mixture and that, crucially, remains compatible with the ESPRIT algebra after a coordinate warp. We give the full derivations for (i), (iv) and (v) because those are where the new mathematics lives.

Contents

1 Notation and the map between the two codebases **2**

2 Shared backbone (unchanged from the prototype) **3**
- 2.1 Model and PGF . . . 3
- 2.2 Random–base line sampling and the sum–of–exponentials structure . . . 3
- 2.3 Global–subspace joint ESPRIT . . . 4

3 Difference 1 — Damped PGF queries ($|t| \le 1$) **4**
- 3.1 The problem with the prototype's query points . . . 4
- 3.2 The damped construction (`_damped_query_parameters`) . . . 4
- 3.3 Why $r_\star = 1 - \log n/(2L_{\text{data}})$? . . . 5

4 Difference 2 — Similarity (non-unitary) joint diagonalisation **5**
- 4.1 Prototype: unitary Cardoso–Souloumiac . . . 5
- 4.2 cikm: similarity diagonalisation (`similarity_joint_diag`) . . . 5

5 Difference 3 — Constrained recovery of rates and weights **6**
- 5.1 Rates: non-negative least squares per direction's spacing . . . 6
- 5.2 Weights: simplex-constrained least squares . . . 6
- 5.3 Conditioning diagnostics . . . 6

6 Difference 4 — L-BFGS refinement with the analytic PGF gradient **6**
6.1 The loss . . . 6
6.2 Analytic gradient (`pgf_gradient_from_raw_params`) . . . 7
6.3 Reparametrisation → box/unconstrained problem (`_pgf_lbfgs_objective`) . . . 7

7 Difference 5 — The NB1 model (between Poisson and Poisson–Gamma) **7**
7.1 The single-coordinate NB1 law . . . 7
7.2 Why NB1 is JESPRIT-compatible but NB2 is not . . . 8
7.3 Dispersion estimation . . . 8
7.4 Feasibility of the warped damping (`_nb1_query_parameters`) . . . 8

8 Likelihood baselines (model-level comparators) **9**

9 Metrics and experimental design **9**
9.1 Error metrics (`metrics.py`) . . . 9
9.2 Sketch-vs-raw budget study (`sketch_vs_raw.py`) . . . 9
9.3 Drivers and configs . . . 10

10 Engineering differences that affect the math **10**

11 Summary of conceptual deltas **10**

1 Notation and the map between the two codebases

Throughout we use the supervisor's notation (which matches his draft and the `cikm` code). Table 1 gives the translation to the symbols used in the original JESPRIT write-up.

Object	`cikm` / this note	Original JESPRIT prototype
Number of mixture components	K	r
Ambient count dimension	d	d
Rate vector of component k	$\boldsymbol{\lambda}_k \in \mathbb{R}_+^d$	$\boldsymbol{\omega}_k$
Rate matrix	$\boldsymbol{\Lambda} = [\boldsymbol{\lambda}_1, \dots, \boldsymbol{\lambda}_K] \in \mathbb{R}_+^{d \times K}$	Ω
Mixture weight	π_k	a_k
Number of probing directions	M	M
Direction vectors	$\boldsymbol{u}_1, \dots, \boldsymbol{u}_M$, $U \in \mathbb{R}^{M \times d}$	$\boldsymbol{u}_l$
Base points (snapshots)	$\boldsymbol{p}_1, \dots, \boldsymbol{p}_S$	$\boldsymbol{p}_{l,s}$
Samples along a line	L (index $\ell = 0, \dots, L-1$)	N
Total sketch size	$J = MSL$	—
Nominal sketch scale	Δ	Δ
Effective scale after damping	$\delta_{\text{eff}} = \alpha\Delta$	(n/a, $\delta_{\text{eff}} = \Delta$)
Damping real part	$r \in (0, 1]$	(n/a, $r = 1$)

Table 1: Symbol dictionary. The prototype always uses $r = 1$, $\delta_{\text{eff}} = \Delta$ implicitly, because its query points are $t = 1 + j\Delta\xi$.

File map of the `cikm` package.

- `core.py` — dataclasses: `MixtureParams`, `SketchConfig`, `PGFSketch`, `EMConfig`, `PGDConfig`, `ExperimentConfig`.
- `pgf.py` — empirical PGF, *damped* line sampling, the *NB1* warp φ and its inverse, the count–table fast path, complex noise injection.

- `jesprit.py` — global SVD, per-direction rotational invariance, joint diagonalisation, phase → rate projection, weight system, diagnostics.
- `linalg.py` — *similarity* joint diagonalisation, simplex projection, `nnls` rate solve, SLSQP simplex weight solve.
- `refinement.py` — L–BFGS–B refinement with the analytic PGF gradient (replaces PGD).
- `em.py`, `poisson_gamma.py`, `poisson_gamma_nb1.py` — the three likelihood baselines (finite Poisson EM, Poisson–Gamma "NB2" EM, NB1 EM).
- `metrics.py`, `experiments.py`, `sketch_vs_raw.py`, `real_data.py`, `cli.py` — evaluation, drivers, real datasets.

2 Shared backbone (unchanged from the prototype)

We recall the common model so the new pieces have a place to attach.

2.1 Model and PGF

The finite product–Poisson mixture is

$$Z \sim \sum_{k=1}^{K} \pi_k \, \delta_{\boldsymbol{\lambda}_k}, \qquad X \mid Z = \boldsymbol{\lambda}_k \sim \bigotimes_{q=1}^{d} \mathrm{Poisson}(\lambda_{qk}), \tag{1}$$

with $\boldsymbol{\lambda}_k \in \mathbb{R}_+^d$, $\pi_k \geq 0$, $\sum_k \pi_k = 1$. Because a Poisson(λ) variable has PGF $\mathbb{E}[t^X] = e^{\lambda(t-1)}$, the mixture PGF is a weighted sum of exponentials,

$$G_X(\boldsymbol{t}) = \mathbb{E}\Big[\textstyle\prod_{q=1}^{d} t_q^{X_q}\Big] = \textstyle\sum_{k=1}^{K} \pi_k \exp\big(\langle \boldsymbol{\lambda}_k, \boldsymbol{t} - \mathbf{1}\rangle\big), \qquad \boldsymbol{t} \in \mathbb{C}^d. \tag{2}$$

The empirical PGF from samples $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(n)}$ is

$$\widehat{G}_X(\boldsymbol{t}) = \frac{1}{n} \sum_{i=1}^{n} \prod_{q=1}^{d} t_q^{x_q^{(i)}} = \frac{1}{n} \sum_{i=1}^{n} \exp\big(\langle \boldsymbol{x}^{(i)}, \log \boldsymbol{t}\rangle\big). \tag{3}$$

This is identical in `empirical_pgf` (cikm) and `PGF` (prototype).

2.2 Random–base line sampling and the sum–of–exponentials structure

Pick M directions $\boldsymbol{u}_m$, S shared base points $\boldsymbol{p}_s$, and sample L regularly spaced points along each line,

$$\boldsymbol{\xi}_{m,s,\ell} = \boldsymbol{p}_s + \ell \, \boldsymbol{u}_m, \qquad m = 1{:}M,\ s = 1{:}S,\ \ell = 0{:}L-1. \tag{4}$$

Evaluating (2) at any query $\boldsymbol{t} = \mathbf{1} + (\text{linear in } \boldsymbol{\xi})$ produces, at the population level, an *exact* multidimensional sum of exponentials:

$$G_X(\boldsymbol{t}_{m,s,\ell}) = \sum_{k=1}^{K} c_{k,s} \, \rho_{m,k}^{\ell}, \qquad \rho_{m,k} = e^{\,i\phi_{m,k}}, \quad \phi_{m,k} = \delta_{\text{eff}} \, \langle \boldsymbol{\lambda}_k, \boldsymbol{u}_m \rangle, \tag{5}$$

with $c_{k,s}$ absorbing π_k and the base–point phase. Stacking the S snapshots of direction m gives $Z_m = V_m C + E_m \in \mathbb{C}^{L \times S}$ with a Vandermonde factor V_m; stacking directions gives the global model $Z_{\text{glob}} = V_{\text{glob}} C + E_{\text{glob}}$. Shared base points make C *common* across directions, which is what lets a single change of basis align all directions.

2.3 Global–subspace joint ESPRIT

Both codebases implement the same four moves, and importantly both use the *global SVD* variant rather than independent per-direction subspaces:

1. Reshape to $Z_{\text{glob}} \in \mathbb{C}^{ML \times S}$ and take its rank-K left singular subspace $\widehat{U}_s$ (`jesprit.py`: `svd(global_matrix)`; prototype: `svd(X_glob)`). Using one global SVD fixes a single basis rotation for all directions, which is the prerequisite for joint diagonalisation.
2. For each direction block $\widehat{U}_{s,m} \in \mathbb{C}^{L \times K}$, solve the rotational invariance equation $\widehat{U}^{\uparrow}_{s,m}\,\Psi_m \approx \widehat{U}^{\downarrow}_{s,m}$ by least squares, $\Psi_m = (\widehat{U}^{\uparrow}_{s,m})^{+}\widehat{U}^{\downarrow}_{s,m}$.
3. Jointly diagonalise $\{\Psi_m\}$ to get a common eigenbasis and the aligned eigenvalues $\mu_{m,k}$.
4. Read phases $\phi_{m,k} = \arg(\mu_{m,k})$, recover rates from $\phi_{m,k} = \delta_{\text{eff}}\langle \boldsymbol{\lambda}_k, \boldsymbol{u}_m\rangle$, and recover weights by least squares against all sketch values.

Everything below is about *how* the supervisor changed the query points (step 0), the diagonaliser (step 3), and the two recovery solves (step 4), plus the new refinement and NB1 layers.

3 Difference 1 — Damped PGF queries ($|\boldsymbol{t}| \le 1$)

3.1 The problem with the prototype's query points

The prototype evaluates the PGF at

$$\boldsymbol{t}_{m,s,\ell} = \mathbf{1} + i\Delta\,\boldsymbol{\xi}_{m,s,\ell} \qquad \Longrightarrow \qquad |t_q| = \sqrt{1 + (\Delta\xi_q)^2} \;>\; 1.$$

Every coordinate lies *outside* the unit circle. In the empirical PGF (3) the contribution of a sample is $\prod_q t_q^{x_q}$, whose modulus is $\prod_q |t_q|^{x_q} = \exp\big(\sum_q x_q \log|t_q|\big)$. With $|t_q| > 1$ this grows *exponentially in the counts*: a single high-count observation can dominate the average and the estimator variance explodes for large Poisson rates. This is exactly the "large rates make PGF evaluations numerically unstable" limitation noted in the draft.

3.2 The damped construction (`_damped_query_parameters`)

The supervisor instead places the real part *strictly inside* the disk and rescales the imaginary excursion so that the whole query family satisfies $|\boldsymbol{t}| \le 1$. Concretely the query points become

$$\boxed{\boldsymbol{t}_{m,s,\ell} = r + i\,\delta_{\text{eff}}\,\boldsymbol{\xi}_{m,s,\ell}} \qquad r \in (0,1], \quad \delta_{\text{eff}} = \alpha\Delta,\ \alpha \in (0,1]. \tag{6}$$

The parameters (r, α) are chosen as follows.

1. **Data scale.** Let $T_i = \sum_q |x_q^{(i)}|$ be the total count of sample i, and set $L_{\text{data}} = \max\big(1,\ \text{quantile}_{0.95}(T)\big)$, a robust upper bound on the count magnitude.
2. **Target radius.** Choose
$$r_\star = 1 - \frac{\log n}{2\,L_{\text{data}}}, \qquad \text{clipped to } (10^{-6},\, 1 - 10^{-6}). \tag{7}$$
3. **Imaginary budget.** Set $b_\star = \sqrt{1 - r_\star^2}$ (so $r_\star^2 + b_\star^2 = 1$), measure the current imaginary extent $b_{\text{old}} = \max_{m,s,\ell} |\Delta\,\boldsymbol{\xi}_{m,s,\ell}|$, and shrink the scale by $\alpha = \min(1,\ b_\star / b_{\text{old}})$.
4. **Final point.** With $b = \min(\alpha\, b_{\text{old}},\ 1 - \varepsilon)$ set $r = \sqrt{1 - b^2}$ and $\delta_{\text{eff}} = \alpha\Delta$.

By construction the most extreme query lies *on* the circle and all others inside it:

$$\max_{m,s,\ell} |\boldsymbol{t}_{m,s,\ell}|^2 = r^2 + (\delta_{\text{eff}}\, b_{\text{old}}/\Delta)^2 \cdot \Delta^2 = r^2 + b^2 = 1.$$

The code asserts $\max|\boldsymbol{t}| \le 1 + 10^{-10}$ and raises otherwise.

3.3 Why $r_\star = 1 - \log n/(2L_{\text{data}})$?

This is the key statistical choice. For a sample with the typical worst-case count L_{data}, the modulus of its PGF term is bounded by

$$r_\star^{L_{\text{data}}} = \Big(1 - \tfrac{\log n}{2L_{\text{data}}}\Big)^{L_{\text{data}}} \;\approx\; \exp\Big(-\tfrac{\log n}{2}\Big) \;=\; \frac{1}{\sqrt{n}}.$$

So the damping is tuned so that the heaviest-count samples are attenuated to exactly the $n^{-1/2}$ stochastic floor of the empirical average (3): damping any harder would push the signal below the Monte-Carlo noise, while damping any less (the prototype's $r = 1$ or $r > 1$) leaves high-count terms with $O(1)$ or exploding modulus and inflated variance. It is the bias–variance sweet spot for the empirical PGF on the unit disk.

Remark 1 (Damping does not break the ESPRIT algebra)**.** Along direction m the consecutive ratio of (5) is $\rho_{m,k} = \exp(i\delta_{\text{eff}}\,\langle \boldsymbol{\lambda}_k, \boldsymbol{u}_m\rangle\,\Delta\ell)$, still of unit modulus. The real part r and the base-point/offset terms only rescale the per-line amplitude $c_{k,s}$ and are absorbed into the right factor C. Hence the poles stay on the unit circle and steps 1–4 are unchanged; only the rate read-out uses δ_{eff} in place of Δ and divides by the per-direction spacing (see §5).

4 Difference 2 — Similarity (non-unitary) joint diagonalisation

4.1 Prototype: unitary Cardoso–Souloumiac

The prototype's `joint_diag` is a direct port of Cardoso's Jacobi-angle routine: it searches for a *unitary* V minimising the total off-diagonal energy of $\{V^H \Psi_m V\}$ via complex Givens rotations. This is optimal when the Ψ_m are *normal* (e.g. Hermitian covariance-like blocks).

4.2 cikm: similarity diagonalisation (`similarity_joint_diag`)

The rotational invariance matrices obey $\Psi_m = T^{-1}\,\mathrm{diag}(\rho_{m,1},\ldots,\rho_{m,K})\,T$ for a *common but generally non-unitary* $T \in \mathrm{GL}(K)$ — the columns of T^{-1} are the (non-orthonormal) component directions in the global subspace. Forcing V unitary is therefore model-mismatched. The supervisor's routine looks for a general similarity instead:

1. Draw a random complex combination $\Psi(\boldsymbol{c}) = \sum_m c_m \Psi_m$ (trial 0 uses $c_m \equiv 1$; later trials use complex Gaussian c_m).
2. Eigendecompose $\Psi(\boldsymbol{c}) = V\,\Lambda\,V^{-1}$; the eigenvectors V are a candidate common basis.
3. Score the candidate by the *off-diagonal energy ratio*

$$\eta(V) = \frac{\sum_m \big\| V^{-1}\Psi_m V - \mathrm{diag}(V^{-1}\Psi_m V)\big\|_F^2}{\sum_m \big\|\,\mathrm{diag}(V^{-1}\Psi_m V)\big\|_F^2}. \tag{8}$$

4. Over 64 random trials keep the V with the smallest η (and report $\mathrm{cond}(V)$ as a diagnostic).

A single generic combination already shares its eigenvectors with every Ψ_m in the noiseless case; the random trials guard against the unlucky event that the chosen combination has near-degenerate eigenvalues (ill-conditioned V). The reported $\mathrm{cond}(V)$ and η feed the convergence flag. The prototype's unitary routine is kept only as a backward-compatible alias.

5 Difference 3 — Constrained recovery of rates and weights

5.1 Rates: non-negative least squares per direction's spacing

After joint diagonalisation we have aligned eigenvalues $\mu_{m,k}$ and phases $\phi_{m,k} = \arg(\mu_{m,k})$. The cikm code first divides out the (possibly direction-specific) sample spacing $\Delta\ell_m$ that it validates from `line_offsets`:

$$\widehat{g}_{m,k} \;=\; \frac{\phi_{m,k}}{\delta_{\text{eff}}\,(\Delta\ell)_m} \;\approx\; \langle \boldsymbol{\lambda}_k, \boldsymbol{u}_m \rangle. \tag{9}$$

Then, instead of the prototype's $\widehat{\boldsymbol{\omega}}_k = \frac{1}{\Delta} U^{+} \boldsymbol{\phi}_k$ followed by $|\mathrm{Re}(\cdot)|$, it solves a genuine non-negative least squares for each component

$$\widehat{\boldsymbol{\lambda}}_k \;=\; \arg\min_{\boldsymbol{\lambda} \geq \mathbf{0}} \; \| U\boldsymbol{\lambda} - \widehat{\boldsymbol{g}}_k \|_2^2 \qquad (\texttt{scipy.optimize.nnls}), \tag{10}$$

which enforces $\boldsymbol{\lambda}_k \in \mathbb{R}_+^d$ exactly rather than by post-hoc absolute value. (The draft's "wrapped least squares" with the integer wrap $\boldsymbol{q}$ degenerates to this when the no-wrapping condition $|\delta_{\text{eff}} \langle \boldsymbol{\lambda}_k, \boldsymbol{u}_m \rangle| < \pi$ holds; the code monitors $\max_{m,k} |\phi_{m,k}|$ against 0.9π and raises a phase-wrapping warning otherwise.)

5.2 Weights: simplex-constrained least squares

The weight system reuses every sketch value. With the linear query rows $\boldsymbol{b}_j = (r-1) + i\delta_{\text{eff}}\,\boldsymbol{\xi}_j \in \mathbb{C}^d$ and design matrix $A_{jk} = \exp(\langle \boldsymbol{b}_j, \boldsymbol{\lambda}_k \rangle)$ (this is $\exp((\boldsymbol{t}_j - 1)\cdot\boldsymbol{\Lambda})$, consistent with (2) under damping), the weights solve

$$\widehat{\boldsymbol{\pi}} = \arg\min_{\boldsymbol{\pi} \in \Delta_K} \; \|A\boldsymbol{\pi} - \boldsymbol{y}\|_2^2, \qquad \Delta_K = \{\boldsymbol{\pi} \geq \mathbf{0} :\; \mathbf{1}^{\top}\boldsymbol{\pi} = 1\}, \tag{11}$$

solved by SLSQP with the analytic objective/gradient $\nabla = 2\,\mathrm{Re}(A^H(A\boldsymbol{\pi} - \boldsymbol{y}))$ and a final Euclidean projection onto Δ_K (`solve_simplex_least_squares` + `project_simplex`). The prototype used $\boldsymbol{a} = \mathrm{pinv}(A)\,\boldsymbol{y}$ then $|\cdot|$ and renormalised — unconstrained and not a true simplex solution.

5.3 Conditioning diagnostics

A genuinely new piece: the code estimates the conditioning of the snapshot factor $C = \mathrm{diag}(\pi)\exp(\boldsymbol{\Lambda}^{\top}[(r-1) + i\delta_{\text{eff}}\,\boldsymbol{p}])$ (the matrix that must be well-conditioned for the weight solve to be identifiable) and reports $\mathrm{cond}(C)$ both at the estimated and (when available) the true parameters, alongside the SVD gap σ_K/σ_{K+1} and the joint-diagonalisation residuals.

6 Difference 4 — L-BFGS refinement with the analytic PGF gradient

This is the item the supervisor described as "PGD replaced by a Python solver using the true analytical gradient, with a more efficient descent method."

6.1 The loss

ESPRIT gives a consistent but algebraic estimate; it does not minimise the PGF fit directly. The refinement minimises the exact (unprojected) PGF least-squares loss over all $J = MSL$ queries,

$$F(\boldsymbol{\Lambda}, \boldsymbol{\pi}) = \frac{1}{2J} \sum_{j=1}^{J} \Big| \underbrace{\textstyle\sum_k \pi_k\, e^{\langle \boldsymbol{b}_j, \boldsymbol{\lambda}_k \rangle}}_{(A\boldsymbol{\pi})_j} - y_j \Big|^2, \qquad \boldsymbol{b}_j = (r-1) + i\delta_{\text{eff}}\,\boldsymbol{\xi}_j. \tag{12}$$

6.2 Analytic gradient (`pgf_gradient_from_raw_params`)

Write $\boldsymbol{\rho} = A\boldsymbol{\pi} - \boldsymbol{y}$ (residual). Treating the loss as a real function of real parameters via $|z|^2 = z\bar{z}$,

$$\frac{\partial F}{\partial \pi_k} = \frac{1}{J}\,\mathrm{Re}\Big(\textstyle\sum_j \overline{A_{jk}}\,\rho_j\Big) = \tfrac{1}{J}\,\mathrm{Re}\big[(A^H\boldsymbol{\rho})_k\big], \tag{13}$$

$$\frac{\partial F}{\partial \lambda_{ik}} = \frac{\pi_k}{J}\,\mathrm{Re}\Big(\textstyle\sum_j \overline{b_{ji}}\,\overline{A_{jk}}\,\rho_j\Big) = \tfrac{\pi_k}{J}\,\mathrm{Re}\big[\big(B^H(\overline{A_{:k}}\odot\boldsymbol{\rho})\big)_i\big], \tag{14}$$

using $\partial A_{jk}/\partial\lambda_{ik} = A_{jk}\,b_{ji}$. Equations (13)–(14) are exactly the closed forms in the code; a finite-difference check is the natural unit test.

6.3 Reparametrisation → box/unconstrained problem (`_pgf_lbfgs_objective`)

To turn the constrained problem ($\boldsymbol{\lambda} \geq 0$, $\boldsymbol{\pi} \in \Delta_K$) into something L-BFGS-B can chew on, the code optimises in transformed coordinates

$$\lambda_{ik} = e^{\theta_{ik}} \quad \text{(positivity)}, \qquad \pi_k = \mathrm{softmax}(\boldsymbol{\eta})_k = \frac{e^{\eta_k}}{\sum_{k'} e^{\eta_{k'}}} \quad \text{(simplex)}. \tag{15}$$

The chain rule gives the transformed gradients actually returned to the optimiser:

$$\frac{\partial F}{\partial \theta_{ik}} = \lambda_{ik}\,\frac{\partial F}{\partial \lambda_{ik}}, \tag{16}$$

$$\frac{\partial F}{\partial \eta_\ell} = \pi_\ell\Big(\frac{\partial F}{\partial \pi_\ell} - \sum_k \pi_k \frac{\partial F}{\partial \pi_k}\Big), \tag{17}$$

the second line being the softmax Jacobian $\partial\pi_k/\partial\eta_\ell = \pi_k(\delta_{k\ell} - \pi_\ell)$ contracted with (13). The θ are box-bounded to $[\log\lambda_{\min}, \log\lambda_{\max}]$; the η are free. `scipy.optimize.minimize` with `method="L-BFGS-B"` and `jac=True` then runs to a gradient/function tolerance.

Why this is better than the old PGD. The old projected-gradient loop (still parametrised by `PGDConfig`: `eta_rates`, `eta_weights`, `momentum`, a diagonal preconditioner, backtracking line search) needed hand-tuned step sizes and a projection onto Δ_K every iteration. The reparametrised L-BFGS-B (a) makes the constraints automatic (no projection), (b) gets curvature information for free through the quasi-Newton Hessian approximation, and (c) needs no learning-rate tuning. `refine_pgf_pgd_from_sketch` is now a thin alias to the L-BFGS routine, so existing call sites and configs keep working.

7 Difference 5 — The NB1 model (between Poisson and Poisson–Gamma)

This is the second item the supervisor flagged: "a distribution model between finite Poisson and Poisson–Gamma that works better on PBMC3k while still being compatible with JESPRIT (NB1)." The PBMC3k single-cell counts are overdispersed, so a pure Poisson mixture underfits; a full Poisson–Gamma mixture fits but *breaks* the ESPRIT algebra. NB1 is the middle ground that keeps the algebra.

7.1 The single-coordinate NB1 law

Let coordinate q be negative-binomial with mean λ_q and a *dimension-wise* dispersion $c_q > 0$ shared by all mixture components, i.e. Gamma shape $= c_q\lambda_q$ and $\mathrm{Var} = \lambda_q(1 + 1/c_q)$. Its PGF is

$$\mathbb{E}\big[t^{X_q}\big] = \Big(1 + \tfrac{1-t}{c_q}\Big)^{-c_q\lambda_q} = \exp\big(\lambda_q\,\varphi_{c_q}(t)\big), \qquad \boxed{\varphi_c(t) = c\,\log\frac{c}{c+1-t}} \tag{18}$$

(`nb1_phi`). As $c \to \infty$, $\varphi_c(t) \to t - 1$ and the law collapses to Poisson; small c is heavy overdispersion. The mean is λ_q for every c, so λ keeps its meaning across the family.

7.2 Why NB1 is JESPRIT-compatible but NB2 is not

The crucial property is that (18) is *linear in the rate* λ_q. For the product mixture,

$$G_X(\boldsymbol{t}) = \sum_{k=1}^{K} \pi_k \prod_{q=1}^{d} \exp\big(\lambda_{qk} \varphi_{c_q}(t_q)\big) = \sum_{k=1}^{K} \pi_k \exp\big(\langle \boldsymbol{\lambda}_k, \boldsymbol{w} \rangle\big), \qquad \boldsymbol{w} := \boldsymbol{\varphi_c}(\boldsymbol{t}), \tag{19}$$

where the warp acts coordinate-wise, $w_q = \varphi_{c_q}(t_q)$. In the *warped variable* $\boldsymbol{w}$ this is identical to the finite-Poisson PGF (2) with $\boldsymbol{t} - \mathbf{1}$ replaced by $\boldsymbol{w}$. Therefore, if we choose the warped query to be the very same damped line, $\boldsymbol{w} = (r-1) + i\delta_{\text{eff}}\,\boldsymbol{\xi}$, and evaluate the empirical PGF at the *pre-image*

$$\boldsymbol{t}^{\text{act}} = \boldsymbol{\varphi_c}^{-1}(\boldsymbol{w}), \qquad \varphi_c^{-1}(w) = c + 1 - c\,e^{-w/c} \tag{20}$$

(`nb1_phi_inv`), then the sketch values inherit *exactly* the sum-of-exponentials structure (5) and the entire JESPRIT pipeline (global SVD, joint diagonalisation, NNLS rates, simplex weights, L-BFGS refinement) applies verbatim. By contrast, a full Poisson–Gamma "NB2" factor $(1 - \frac{\lambda}{s}(t-1))^{-s}$ with a free per-component shape s has log-PGF $-s\log(1 - \frac{\lambda}{s}(t-1))$, which is *not* linear in λ and admits no component-independent warp — so NB2 cannot be folded into ESPRIT.

7.3 Dispersion estimation

The dimension-wise c_q is estimated once by moments (`estimate_nb1_dispersion_mom`): matching $\mathrm{Var} = \lambda(1 + 1/c)$ to the empirical mean/variance gives

$$c_q = \frac{\overline{X_q}}{\widehat{\mathrm{Var}}(X_q) - \overline{X_q}}, \tag{21}$$

clipped to $[10^{-3}, 10^{6}]$; non-overdispersed coordinates ($\widehat{\mathrm{Var}} \le \overline{X}$) get $c_q = c_{\max}$, i.e. effectively Poisson.

7.4 Feasibility of the warped damping (`_nb1_query_parameters`)

The one subtlety: the empirical PGF must still be queried at points with $|\boldsymbol{t}^{\text{act}}| \le 1$. Write $w = a + iy$ with real shift $a = r - 1 \le 0$ and $y = \delta_{\text{eff}}\,\xi$. With $\rho := e^{-a/c} \ge 1$,

$$t^{\text{act}} = (c+1) - c\rho\cos(y/c) + i\,c\rho\sin(y/c), \qquad |t^{\text{act}}|^2 = (c+1)^2 - 2c(c+1)\rho\cos(y/c) + c^2\rho^2.$$

Imposing $|t^{\text{act}}|^2 \le 1$ is a quadratic in ρ,

$$c\,\rho^2 - 2(c+1)\cos(y/c)\,\rho + (c+2) \;\le\; 0, \tag{22}$$

feasible iff the discriminant $D = (c+1)^2\cos^2(y/c) - c(c+2) \ge 0$, with admissible band

$$\rho \in [\rho_-, \rho_+], \qquad \rho_\pm = \frac{(c+1)\cos(y/c) \pm \sqrt{D}}{c}. \tag{23}$$

Since $\rho = e^{-a/c} \ge 1$ we clip $\rho_- \leftarrow \max(\rho_-, 1)$ and translate back to the real shift $a = -c\log\rho \in [-c\log\rho_+,\, -c\log\rho_-]$. Taking the tightest bound across all coordinates and queries gives a feasible interval $[a_{\text{lower}}, a_{\text{upper}}]$; the code picks $r = 1 + a_{\text{upper}}$ (the least damping), and if no feasible a exists it halves α (shrinks δ_{eff}) and retries, up to 80 times. This is the exact loop in `_nb1_query_parameters`; the resulting $\boldsymbol{t}^{\text{act}}$ are stored in the sketch as `actual_query_points`.

8 Likelihood baselines (model-level comparators)

The cikm package ships three hand-written EM fitters, used as comparators (not as part of the sketching pipeline). All share the standard product-mixture E-step (responsibilities via `logsumexp`) and differ only in the per-coordinate emission law and its M-step.

File / fitter	Emission law	M-step specifics
`em.py` `fit_poisson_em`	Product Poisson, Var = mean	closed-form $\lambda_{qk} = \frac{\sum_i r_{ik} x_q^{(i)}}{\sum_i r_{ik}}$.
`poisson_gamma.py` `fit_poisson_gamma_em`	"NB2": per-(coord,comp) free Gamma shape s_{qk}, Var $= \mu + \mu^2/s$	mean closed-form; each s_{qk} by a 1-D bounded Newton/Brent on the responsibility-weighted NB log-likelihood (MoM warm start).
`poisson_gamma_nb1.py` `fit_poisson_gamma_nb1_em`	"NB1": dimension-wise dispersion c_q shared across components, shape $= c_q \mu_{qk}$	mean closed-form; each c_q by a 1-D bounded search on $\log c$ over all components of coordinate q.

Table 2: The three EM baselines. NB1 here is the *likelihood* counterpart of the warped sketch model of §7, sharing the same Var $= \mu(1 + 1/c)$ parametrisation.

The NB2 log-pmf uses shape $\cdot \log \frac{\text{shape}}{\text{shape}+\mu} + x \log \frac{\mu}{\text{shape}+\mu}$ with free shape; the NB1 log-pmf substitutes shape $= c\mu$ and simplifies to $c\mu \log \frac{c}{c+1} - x \log(c+1)$ plus the Gamma-ratio terms.

9 Metrics and experimental design

9.1 Error metrics (`metrics.py`)

- **Component matching** by the Hungarian algorithm on a rate+weight cost (same idea as the prototype's `compute_error`, but the cost is normalised by $\max(|\lambda|, 1)$ and includes an explicit penalty for unmatched components when $K_{\text{est}} \neq K_{\text{true}}$).
- **Rate / weight relative error** after matching.
- **PGF reconstruction error** $\|\widehat{G} - G_{\text{model}}\| / \|\widehat{G}\|$ on a held-out query set.
- **Distributional fit**: per-coordinate *Hellinger* and a stabilised χ^2 between the empirical marginal histogram and the model marginal pmf, averaged over coordinates.

9.2 Sketch-vs-raw budget study (`sketch_vs_raw.py`)

This is the headline scalability argument. Raw storage is nd scalars; the sketch is a *fixed* $2J = 2MSL$ real scalars (real+imag), independent of n. The driver compares, at a *matched storage budget*:

- *Full raw EM* (uses all nd scalars) — the accuracy ceiling;
- *Raw subset EM* on $m \approx 2J/d$ samples — EM at the same storage as the sketch;
- *PGF-JESPRIT* and *PGF-JESPRIT + L-BFGS* — from $2J$ sketch scalars.

A separate n-sweep holds J fixed and varies $n \in [10^3, 3{\cdot}10^5]$, decomposing runtime into the $O(n)$ sketch-construction step and the n-independent recovery step — the plot that shows recovery cost flat in n while EM grows.

9.3 Drivers and configs

`experiments.py` runs the synthetic grid (regimes *separated*, *overlap*, *underfit*, *rare_large_count*), a Δ-sensitivity sweep, and the real-data study (Affairs, PBMC3k) wiring all methods — including *NB1-PGF-JESPRIT* and the three EM baselines — through a shared sketch. The YAML configs in `configs/` (e.g. `real_cikm2026.yaml`) select methods, sketch offsets M, S, L, `delta_scale`, and NB1 dispersion bounds.

10 Engineering differences that affect the math

- **Count-table fast path** (`_sample_pgf_lines_from_count_tables`). For integer data, the term $\prod_q t_q^{x_q}$ is computed from precomputed power tables $\{t_q^0, \dots, t_q^{\max}\}$ and (optionally) duplicate-row compression, turning the $O(nJd)$ empirical PGF into table look-ups. Numerically identical to the naive sum, just far faster on real count matrices. The prototype only had the batched naive sum (with `joblib` threading).
- **Complex sketch noise** (`_add_complex_noise`). Optional circular Gaussian noise at a target SNR is injected into the sketch values so robustness to sketch error can be studied; powers (clean / noise / total) are recorded.
- **Per-direction spacing validation** (`_direction_spacings`). The code verifies each ESPRIT row is a regular grid shared across snapshots and extracts the spacing $(\Delta\ell)_m$ used in the phase→rate division; non-zero offset starts are allowed and absorbed into C.
- **Rich diagnostics**. Every fit returns SVD gap, cond(U), cond(V), joint off-diagonal ratio, Ψ residuals, max phase vs. 0.9π, projection residuals, snapshot-factor conditioning, and timing split into sketch / recovery / refinement. The prototype returned only point estimates.

11 Summary of conceptual deltas

Stage	Prototype (`JESPRIT/src`)	cikm (`src/pgf_jesprit`)
PGF query point	$\boldsymbol{t} = \mathbf{1} + i\Delta\boldsymbol{\xi}$, $\lvert\boldsymbol{t}\rvert > 1$	$\boldsymbol{t} = r + i\delta_{\text{eff}}\boldsymbol{\xi}$, $\lvert\boldsymbol{t}\rvert \le 1$, with $r = 1 - \frac{\log n}{2L_{\text{data}}}$ (§3)
Subspace	global SVD	global SVD (same)
Joint diag.	unitary Cardoso–Souloumiac Jacobi	similarity (random-combination eigendecomp., off-diag. energy selection)
Rate recovery	$\frac{1}{\Delta}U^+\boldsymbol{\phi}$ then |Re|	per-spacing phase → `nnls` ($\boldsymbol{\lambda} \ge 0$) (§5)
Weight recovery	pinv(A)$\boldsymbol{y}$, $\lvert\cdot\rvert$, renormalise	simplex-constrained LS (SLSQP) (§5)
Refinement	none	L-BFGS-B on exact PGF loss, analytic gradient, log/softmax reparam (§6)
Distribution	finite Poisson only	finite Poisson *and* NB1 warp (§7)
Baselines	—	Poisson EM, NB2 PG EM, NB1 EM
Phase unwrapping	optional, found to hurt	replaced by a 0.9π wrapping *warning*
Diagnostics	none	conditioning, gaps, residuals, timings

Table 3: Where the supervisor's code diverges from the prototype. The three rows in bold-math sections (§3, §6, §7) are the substantive new derivations.

One-line takeaways.

1. **Damping** (§3) is the fix for the prototype's large-rate instability: keep $|\boldsymbol{t}| \leq 1$ and tune r so the heaviest counts hit the $n^{-1/2}$ noise floor.
2. **L-BFGS refinement** (§6) closes the gap between "algebraically consistent" and "actually minimises the PGF loss", using the closed-form gradient and constraint-free coordinates — hence the supervisor's "good performance after the GD step".
3. **NB1** (§7) is the only overdispersion model whose log-PGF is *linear in the rate*; the coordinate warp φ_c maps it back onto the exact sum-of-exponentials that ESPRIT needs, which is why it fits PBMC3k yet stays JESPRIT-compatible.